

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- Implementation of SCD Type 2 for order summary tracking
- Incremental processing triggered by customer updates
- Generation of customer aggregate spend analytics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Data visualization or BI tool integration
- Real-time streaming requirements
- Data archival or retention policies
- User access management or IAM policies

Assumptions

1. CSV files in the source S3 paths are well-formed and use standard delimiters
2. The `Date` field in the Order dataset is in a parseable date format
3. `CustId` is the primary key for Customer data and foreign key in Order data
4. `OrderId` is the primary key for Order data
5. The term "Null" (string) and NULL (actual null value) both represent missing data
6. Duplicate detection is based on all columns in each dataset

7. SCD2 changes are triggered by any change in customer attributes (Name, EmailId, Region)
8. The `OpTs` field represents the operation timestamp for SCD2 tracking
9. AWS Glue job has appropriate IAM permissions to read/write S3 and access Glue Catalog
10. Apache Hudi is configured with appropriate precombine and record key fields
11. TotalAmount in aggregation is calculated as `PricePerUnit Qty` summed per customer
12. The aggregation table reflects the most recent date of customer transactions

-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3

- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a structured format for downstream processing.

- Business Objective:

Enable the ETL pipeline to access customer master data for order enrichment and SCD2 processing.

- Inputs:

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/

- Format: CSV

- Schema:

- `CustId` (data type not specified in BRD)
- `Name` (data type not specified in BRD)
- `EmailId` (data type not specified in BRD)
- `Region` (data type not specified in BRD)

- Processing Rules / Functional Steps:

1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/customerdata/
2. Read all CSV files present in the path
3. Parse CSV content using standard CSV parsing rules
4. Load data into a DataFrame or equivalent structure with columns: CustId, Name, EmailId, Region

- Outputs:

- In-memory customer dataset with schema: CustId, Name, EmailId, Region

- Preconditions:

- S3 path s3://adif-sdlc/sdlc_wizard/customerdata/ exists and contains at least one CSV file
- AWS Glue job has read permissions on the S3 bucket
- Postconditions:
- Customer data is available in memory for data cleaning operations
- Error / Validation Conditions:
- If S3 path does not exist, raise error: "Source path not found"
- If no CSV files are present, raise error: "No customer data files found"
- If CSV parsing fails, raise error: "Invalid CSV format in customer data"
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a structured format for downstream processing.

- Business Objective:

Enable the ETL pipeline to access order transaction data for customer order analysis and aggregation.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- `OrderId` (data type not specified in BRD)
- `ItemName` (data type not specified in BRD)
- `PricePerUnit` (data type not specified in BRD)
- `Qty` (data type not specified in BRD)
- `Date` (data type not specified in BRD)
- `CustId` (data type not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files present in the path
- 3. Parse CSV content using standard CSV parsing rules
- 4. Load data into a DataFrame or equivalent structure with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Outputs:
- In-memory order dataset with schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Preconditions:
- S3 path s3://adif-sdlc/sdlc_wizard/orderdata/ exists and contains at least one CSV file
- AWS Glue job has read permissions on the S3 bucket
- Postconditions:
- Order data is available in memory for data cleaning operations
- Error / Validation Conditions:
- If S3 path does not exist, raise error: "Source path not found"
- If no CSV files are present, raise error: "No order data files found"
- If CSV parsing fails, raise error: "Invalid CSV format in order data"
-

FR-CLEAN-001: Remove Null Values and Duplicates from Customer Data

- Title: Clean Customer Dataset
- Description:

The system shall remove all records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the customer dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant customer records before catalog registration and downstream processing.

- Inputs:
- In-memory customer dataset from FR-INGEST-001
- Processing Rules / Functional Steps:
 1. Identify and remove all rows where any column contains the string "Null" (case-sensitive)
 2. Identify and remove all rows where any column contains actual NULL values
 3. Identify duplicate rows based on all columns (CustId, Name, EmailId, Region)
 4. Retain only the first occurrence of each duplicate set
 5. Return cleaned customer dataset
- Outputs:
- Cleaned customer dataset with no null values and no duplicates

- Preconditions:
- Customer data has been successfully loaded (FR-INGEST-001 completed)
- Postconditions:
- Customer dataset contains only valid, unique records
- Error / Validation Conditions:
- If all records are removed due to null values, log warning: "All customer records contained null values"
- Log the count of records removed due to nulls and duplicates
-

FR-CLEAN-002: Remove Null Values and Duplicates from Order Data

- Title: Clean Order Dataset
- Description:

The system shall remove all records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the order dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant order records before catalog registration and downstream processing.

- Inputs:
- In-memory order dataset from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. Identify and remove all rows where any column contains the string "Null" (case-sensitive)
 2. Identify and remove all rows where any column contains actual NULL values
 3. Identify duplicate rows based on all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)
 4. Retain only the first occurrence of each duplicate set
 5. Return cleaned order dataset
- Outputs:
- Cleaned order dataset with no null values and no duplicates
- Preconditions:
- Order data has been successfully loaded (FR-INGEST-002 completed)
- Postconditions:

- Order dataset contains only valid, unique records
- Error / Validation Conditions:
 - If all records are removed due to null values, log warning: "All order records contained null values"
 - Log the count of records removed due to nulls and duplicates
-

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Enable SQL-based querying of customer data through AWS Athena and maintain metadata for data governance.

- Inputs:
 - Cleaned customer dataset from FR-CLEAN-001
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: sdhc_wizard_customer
- Processing Rules / Functional Steps:
 1. Connect to AWS Glue Data Catalog
 2. Verify database "gen_ai_poc_databrickscoe" exists; create if not present
 3. Create or update table "sdhc_wizard_customer" with schema: CustId, Name, EmailId, Region
 4. Register table metadata including column names, data types, and storage location
 5. Set table properties to enable Athena querying
- Outputs:
 - Glue Catalog table: gen_ai_poc_databrickscoe.sdhc_wizard_customer
- Preconditions:
 - Customer data has been cleaned (FR-CLEAN-001 completed)
 - AWS Glue job has permissions to create/update Glue Catalog tables
- Postconditions:
 - Customer table is queryable via AWS Athena

- Table metadata is available in Glue Data Catalog
- Error / Validation Conditions:
 - If database creation fails, raise error: "Unable to create Glue database"
 - If table registration fails, raise error: "Unable to register customer table in Glue Catalog"
-

FR-CATALOG-002: Register Order Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Enable SQL-based querying of order data through AWS Athena and maintain metadata for data governance.

- Inputs:
 - Cleaned order dataset from FR-CLEAN-002
 - Target Database: gen_ai_poc_databricksco
 - Target Table: sdlc_wizard_order
- Processing Rules / Functional Steps:
 1. Connect to AWS Glue Data Catalog
 2. Verify database "gen_ai_poc_databricksco" exists; create if not present
 3. Create or update table "sdlc_wizard_order" with schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
 4. Register table metadata including column names, data types, and storage location
 5. Set table properties to enable Athena querying
- Outputs:
 - Glue Catalog table: gen_ai_poc_databricksco.sdlc_wizard_order
- Preconditions:
 - Order data has been cleaned (FR-CLEAN-002 completed)
 - AWS Glue job has permissions to create/update Glue Catalog tables
- Postconditions:
 - Order table is queryable via AWS Athena

- Table metadata is available in Glue Data Catalog
- Error / Validation Conditions:
 - If database creation fails, raise error: "Unable to create Glue database"
 - If table registration fails, raise error: "Unable to register order table in Glue Catalog"
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with SCD Type 2 Logic
- Description:

The system shall join customer and order data, implement Slowly Changing Dimension Type 2 logic to track historical changes, and store the result in Apache Hudi format on S3.

- Business Objective:

Maintain a complete historical record of customer-order relationships, capturing changes over time while preserving the ability to query data as it existed at any point in history.

- Inputs:
 - Cleaned customer dataset from FR-CLEAN-001
 - Cleaned order dataset from FR-CLEAN-002
 - Target Table: ordersummary
 - Target Database: gen_ai_poc_databricksco
 - Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Perform inner join between customer and order datasets on CustId
 2. For each joined record, check if a matching active record exists in the ordersummary table
 3. If no matching active record exists (new record):
 - Insert new record with IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
 4. If matching active record exists and customer attributes (Name, EmailId, Region) have changed:
 - Update existing active record: set IsActive = false, EndDate = current timestamp, OpTs = current timestamp
 - Insert new record with updated values: IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
 5. If matching active record exists and no changes detected:
 - No action required (record remains unchanged)

6. Write results to S3 using Apache Hudi format at
s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

7. Register or update ordersummary table in Glue Data Catalog

- Outputs:
- Hudi table stored at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
- Table includes columns: all customer fields, all order fields, IsActive, StartDate, EndDate, OpTs
- Preconditions:
- Customer and order data have been cleaned (FR-CLEAN-001, FR-CLEAN-002 completed)
- S3 path s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ is accessible
- Apache Hudi libraries are available in the Glue job environment
- Postconditions:
- All current and historical customer-order relationships are stored with SCD2 tracking
- Active records have IsActive = true and EndDate = null
- Closed records have IsActive = false and EndDate populated
- Error / Validation Conditions:
- If join produces no results, log warning: "No matching customer-order records found"
- If Hudi write fails, raise error: "Unable to write to Hudi table"
- If Glue Catalog registration fails, raise error: "Unable to register ordersummary table"
-

FR-INCR-001: Implement Incremental Processing for Customer Updates

- Title: Trigger SCD2 Updates Based on Customer Changes

- Description:

The system shall detect changes in customer data and trigger incremental updates to the ordersummary table, maintaining SCD Type 2 history without full table reprocessing.

- Business Objective:

Optimize ETL performance by processing only changed customer records while maintaining complete historical accuracy in the order summary.

- Inputs:
- New or updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/
- Existing ordersummary Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Identify customer records that are new or have changes in Name, EmailId, or Region compared to the last processing run

2. For each changed customer record:

- Retrieve all active order summary records associated with that CustId
- Apply SCD2 logic as defined in FR-SCD2-001 (close old records, insert new active records)
- Update IsActive, StartDate, EndDate, and OpTs fields accordingly

3. Perform incremental upsert to the Hudi table using Hudi's merge capabilities

4. Update Glue Catalog metadata if schema changes occurred

- Outputs:

- Updated ordersummary Hudi table with new SCD2 versions for affected customers
- Updated Glue Catalog metadata

- Preconditions:

- Initial ordersummary table exists (FR-SCD2-001 completed at least once)
- Customer data ingestion has occurred (FR-INGEST-001 completed)

- Postconditions:

- All customer changes are reflected in ordersummary with proper SCD2 tracking
- Historical records remain intact and queryable

- Error / Validation Conditions:

- If no customer changes detected, log info: "No customer updates to process"
- If incremental merge fails, raise error: "Incremental SCD2 update failed"

-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table

- Description:

The system shall aggregate order data by customer to calculate total spending and store the results in a dedicated analytics table on S3.

- Business Objective:

Provide business users with a pre-aggregated view of customer spending for reporting and analysis purposes.

- Inputs:

- Cleaned customer dataset from FR-CLEAN-001

- Cleaned order dataset from FR-CLEAN-002
- Target Table: customeraggregatespend
- Target Database: gen_ai_poc_databricksco
- Storage Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. For each customer, calculate TotalAmount = SUM(PricePerUnit Qty) across all orders
 3. For each customer, retrieve the most recent Date from their orders
 4. Create aggregated dataset with columns: Name, TotalAmount, Date
 5. Write aggregated data to S3 at s3://adif-sdlc/analytics/customeraggregatespend/
 6. Register or update customeraggregatespend table in Glue Data Catalog
- Outputs:
 - Analytics table stored at s3://adif-sdlc/analytics/customeraggregatespend/
 - Glue Catalog table: gen_ai_poc_databricksco.customeraggregatespend
- Schema:
 - `Name` (data type not specified in BRD)
 - `TotalAmount` (data type not specified in BRD)
 - `Date` (data type not specified in BRD)
- Preconditions:
 - Customer and order data have been cleaned (FR-CLEAN-001, FR-CLEAN-002 completed)
 - S3 path s3://adif-sdlc/analytics/customeraggregatespend/ is accessible
- Postconditions:
 - Customer aggregate spend data is available for querying via Athena
 - Table contains one row per customer with total spend and latest transaction date
- Error / Validation Conditions:
 - If join produces no results, raise error: "No customer-order matches found for aggregation"
 - If TotalAmount calculation fails due to non-numeric values, raise error: "Invalid numeric data in PricePerUnit or Qty"
 - If S3 write fails, raise error: "Unable to write aggregate table to S3"
 - If Glue Catalog registration fails, raise error: "Unable to register customeraggregatespend table"
-

FR-CATALOG-003: Register Order Summary Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Summary

- Description:

The system shall register the ordersummary Hudi table in the AWS Glue Data Catalog to enable querying via Athena.

- Business Objective:

Enable SQL-based access to historical order summary data with SCD2 tracking for business intelligence and reporting.

- Inputs:

- ordersummary Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Database: gen_ai_poc_databricksco
- Target Table: ordersummary

- Processing Rules / Functional Steps:

1. Connect to AWS Glue Data Catalog
2. Verify database "gen_ai_poc_databricksco" exists
3. Create or update table "ordersummary" with Hudi-specific properties
4. Register table schema including all customer fields, order fields, and SCD2 tracking fields (IsActive, StartDate, EndDate, OpTs)
5. Configure table properties to support Hudi time-travel queries

- Outputs:

- Glue Catalog table: gen_ai_poc_databricksco.ordersummary

- Preconditions:

- ordersummary Hudi table has been created (FR-SCD2-001 completed)

- Postconditions:

- ordersummary table is queryable via AWS Athena with Hudi support
- Historical queries can be performed using SCD2 tracking fields

- Error / Validation Conditions:

- If table registration fails, raise error: "Unable to register ordersummary table in Glue Catalog"

•

FR-CATALOG-004: Register Customer Aggregate Spend Table in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Aggregate Spend

- Description:

The system shall register the customeragggregatespend table in the AWS Glue Data Catalog to enable querying via Athena.

- Business Objective:

Enable SQL-based access to customer spending analytics for business reporting and decision-making.

- Inputs:

- customeragggregatespend table from s3://adif-sdlc/analytics/customeragggregatespend/

- Target Database: gen_ai_poc_databrickscoe

- Target Table: customeragggregatespend

- Processing Rules / Functional Steps:

1. Connect to AWS Glue Data Catalog
2. Verify database "gen_ai_poc_databrickscoe" exists
3. Create or update table "customeragggregatespend" with schema: Name, TotalAmount, Date
4. Register table metadata and storage location
5. Set table properties to enable Athena querying

- Outputs:

- Glue Catalog table: gen_ai_poc_databrickscoe.customeragggregatespend

- Preconditions:

- customeragggregatespend table has been created (FR-AGG-001 completed)

- Postconditions:

- customeragggregatespend table is queryable via AWS Athena

- Business users can access customer spending analytics

- Error / Validation Conditions:

- If table registration fails, raise error: "Unable to register customeragggregatespend table in Glue Catalog"

-

3. Non-Functional Considerations

Performance

- The ETL pipeline should leverage AWS Glue's distributed Spark processing for scalability
- Incremental processing (FR-INCR-001) should be used to minimize processing time for large datasets

Monitoring and Observability

- AWS CloudWatch should be used to monitor Glue job execution, failures, and performance metrics
- Job logs should be retained for troubleshooting and audit purposes

Data Governance

- AWS Lake Formation may optionally be used for fine-grained access control and data governance
- All tables registered in Glue Data Catalog should have appropriate metadata and descriptions

Reliability

- Glue jobs should implement retry logic for transient failures
- Failed jobs should generate alerts via CloudWatch for operational awareness

Storage Optimization

- Apache Hudi should be configured with appropriate compaction and cleaning strategies to optimize storage costs
- S3 lifecycle policies may be applied to archive or delete old data based on retention requirements
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3) - Customer CSV	Customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Source Input (AWS S3) - Order CSV	Order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Data Cleaning	Remove nulls and duplicates from customer data	
FR-CLEAN-002	Data Cleaning	Remove nulls and duplicates from order data	
FR-CATALOG-001	Glue Catalog Tables	Register sdlc_wizard_customer table	
FR-CATALOG-002	Glue Catalog Tables	Register sdlc_wizard_order table	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Implement SCD2 for ordersummary table	
FR-INCR-001	Incremental Logic	Trigger SCD2 updates on customer changes	
FR-AGG-001	Aggregation Table	Generate customeraggregatespend analytics	
FR-CATALOG-003	Glue Catalog Tables	Register ordersummary table	
FR-CATALOG-004	Glue Catalog Tables	Register customeraggregatespend table	

- Key Traceability Principles:
- All functional requirements are directly derived from explicit BRD content
- Source S3 paths are preserved exactly as specified in the BRD
- Table names and database names match BRD specifications
- SCD2 logic and incremental processing requirements are traced to BRD sections 5 and 6
- No functional requirements were invented beyond what is stated in the BRD